

SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence

COURSE CODE:CSA1709

Register Number	192524317
Name	S DHANUSH

COURSE OUTCOME COVERED

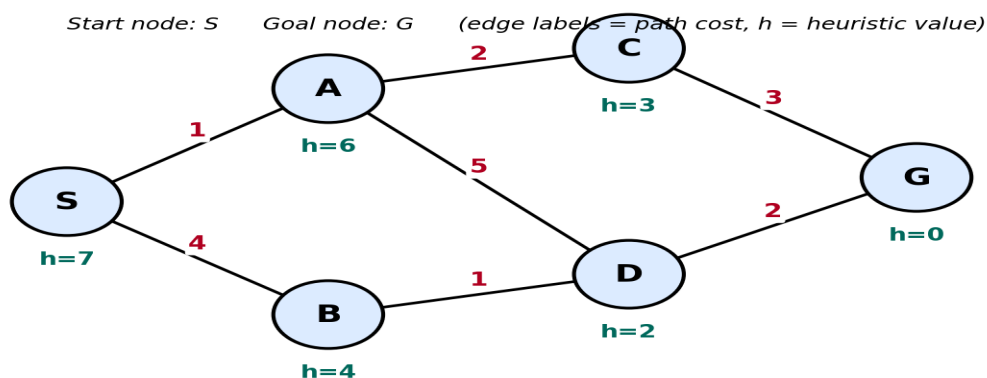
CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. Consider a suitable state-space graph (with heuristic values $h(n)$ assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



Node	Heuristic $h(n)$
S	7
A	6
B	4
C	3
D	2
G	0

Edge Costs (Not used for node selection in Greedy Best-First Search)

- $S \rightarrow A = 1$
- $S \rightarrow B = 4$
- $A \rightarrow C = 2$
- $A \rightarrow D = 5$
- $B \rightarrow D = 1$
- $C \rightarrow G = 3$
- $D \rightarrow G = 2$

Rule: Greedy Best-First Search always expands the node with the **smallest heuristic value $h(n)$** from the frontier (Open List). It ignores the path cost.

Step 1

Current Node

S

Node $h(n)$

S 7

Expand S.

Its successors are:

- A ($h = 6$)
- B ($h = 4$)

Open List after expansion

Node $h(n)$

B 4

A 6

Closed List

S

Step 2

Current Node

B

Open List before expansion

Node h(n)

B 4

A 6

Expand **B**.

Successor:

- D ($h = 2$)

Open List after expansion

Node h(n)

D 2

A 6

Closed List

S, B

Step 3

Current Node

D

Open List before expansion

Node h(n)

D 2

A 6

Expand **D**.

Successor:

- G ($h = 0$)

Open List after expansion

Node h(n)

G 0

Node h(n)

A 6

Closed List

S, B, D

Step 4

Current Node

G

Since **G is the Goal Node**, the search stops.

Search Process Summary

Step	Expanded Node	Open List (after expansion)	Next Selected
1	S	B(4), A(6)	B
2	B	D(2), A(6)	D
3	D	G(0), A(6)	G
4	G	Goal Found	Stop

Final Path Obtained

S → B → D → G

Path Cost (Calculated After Goal is Found)

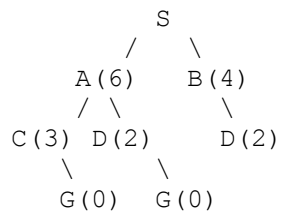
Although GBFS **does not use path cost for selecting nodes**, the final path cost is:

$S \rightarrow B = 4 \quad S \rightarrow B = 4 \quad B \rightarrow D = 1 \quad B \rightarrow D = 1 \quad D \rightarrow G = 2$
 $\rightarrow G = 2 \quad \text{Total Cost} = 4 + 1 + 2 = 7$
 $\text{Total Cost} = 4 + 1 + 2 = 7$

Expansion Order

S
↓
B
↓
D
↓
G

Search Tree

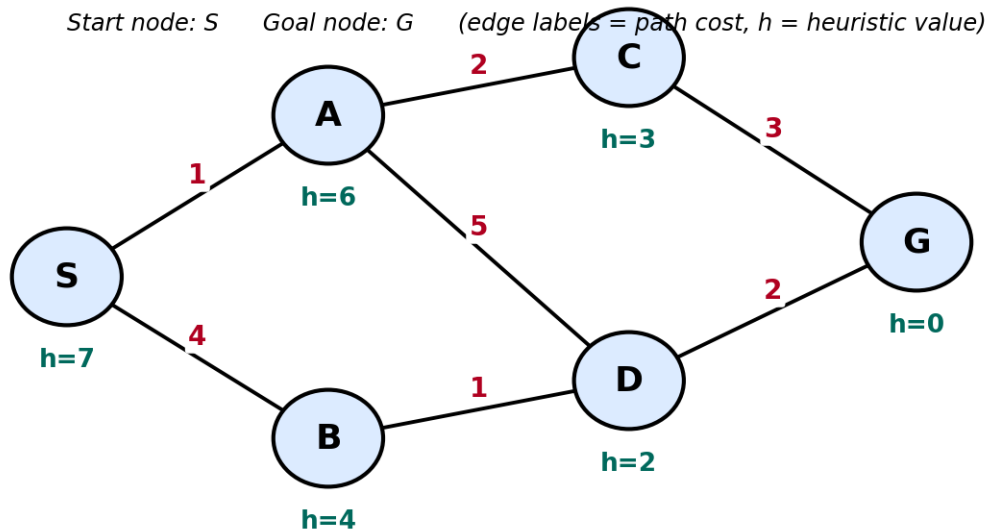


Expanded Nodes:

S → B → D → G

- **Expansion Order:** S → B → D → G
- **Final Path:** S → B → D → G
- **Total Path Cost:** 7
- **Reason:** At every step, Greedy Best-First Search expands the node with the smallest heuristic value from the Open List.

Q2. For a suitable weighted state-space graph (with edge costs and heuristic values $h(n)$) of your choice, perform a dry run of A* Search from the start node to the goal node. Show the $g(n)$, $h(n)$ and $f(n)$ values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



Heuristic Values

Node	$h(n)$
S	7
A	6
B	4
C	3
D	2
G	0

Edge Costs

- $S \rightarrow A = 1$
- $S \rightarrow B = 4$
- $A \rightarrow C = 2$
- $A \rightarrow D = 5$
- $B \rightarrow D = 1$
- $C \rightarrow G = 3$
- $D \rightarrow G = 2$

Formula Used in A*

A* evaluates every node using

$$f(n)=g(n)+h(n) \quad f(n)=g(n)+h(n) \quad f(n)=g(n)+h(n)$$

Where

- **$g(n)$** = Actual path cost from Start node to current node.
- **$h(n)$** = Estimated cost from current node to Goal.
- **$f(n)$** = Estimated total cost.

Initial State

Start at **S**

Calculate values

$$g(S)=0 \quad g(S)=0 \quad g(S)=0 \quad h(S)=7 \quad h(S)=7 \quad h(S)=7 \quad f(S)=0+7=7 \quad f(S)=0+7=7 \quad f(S)=0+7=7$$

OPEN List

Node	g	h	f
S	0	7	7

CLOSED List = Empty

Step 1

Expand **S**

Children are

- A
- B

Node A

Path

$S \rightarrow A$

Edge cost

1

$$g(A)=0+1=1 \quad g(A)=0+1=1 \quad g(A)=0+1=1 \quad h(A)=6 \quad h(A)=6 \quad h(A)=6 \\ f(A)=1+6=7 \quad f(A)=1+6=7 \quad f(A)=1+6=7$$

Node B

Path

$S \rightarrow B$

Edge cost

4

$g(B)=0+4=4$
 $h(B)=4$
 $f(B)=4+4=8$

OPEN List

Node g h f

A 1 6 7

B 4 4 8

Choose smallest **f**

Expand **A**

Step 2

Expand **A**

Children

- C
- D

Node C

Path

$S \rightarrow A \rightarrow C$

$g(C)=1+2=3$
 $h(C)=3$
 $f(C)=3+3=6$

Node D

Path

$S \rightarrow A \rightarrow D$

$g(D)=1+5=6$
 $h(D)=2$
 $f(D)=6+2=8$

OPEN List

Node g h f

C 3 3 6

B 4 4 8

D 6 2 8

Choose minimum f

Expand C

Step 3

Expand C

Child

- G

Node G

Path

$S \rightarrow A \rightarrow C \rightarrow G$

$g(G)=3+3=6$
 $h(G)=0$
 $f(G)=6+0=6$

OPEN List

Node g h f

G 6 0 6

B 4 4 8

D 6 2 8

Minimum is

G

Since G is the goal node,

Stop.

Expansion Order

S
↓
A
↓
C
↓
G

Manual Calculation Table

Step	Expanded Node	Successor	g(n)	h(n)	f(n) = g + h
1	S	S	0	7	7
		A	1	6	7
		B	4	4	8
2	A	C	3	3	6
		D	6	2	8
3	C	G	6	0	6
4	G	Goal Reached	-	-	-

OPEN List After Every Expansion

Initial

S(7)

After Expanding S

OPEN

A(7)

B(8)

After Expanding A

OPEN

C(6)

B(8)

D(8)

After Expanding C

OPEN

G(6)

B(8)

D(8)

After Expanding G

Goal Found

Final Optimal Path

S
|
1
▼
A
|
2
▼
C
|
3
▼
G

Path:

$S \rightarrow A \rightarrow C \rightarrow G$

Total Path Cost

$$1+2+3=6$$

$$1+2+3=6$$

$$1+2+3=6$$

Therefore,

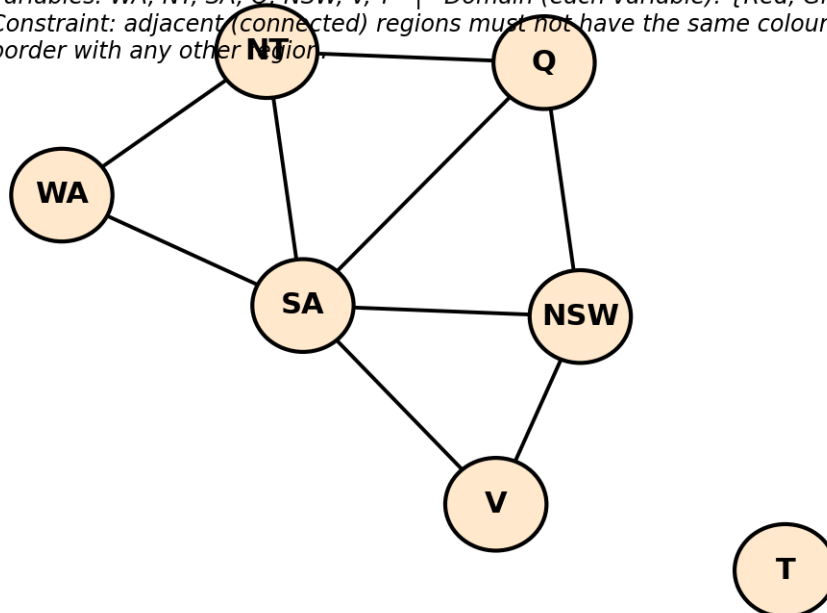
Optimal Cost = 6

Final Answer

- **Algorithm Used:** A* Search
- **Expansion Order:** $S \rightarrow A \rightarrow C \rightarrow G$
- **Optimal Path:** $S \rightarrow A \rightarrow C \rightarrow G$
- **Total Path Cost:** 6
- **Evaluation Function:** $f(n)=g(n)+h(n)$ $f(n)=g(n)+h(n)$ $f(n)=g(n)+h(n)$
- **Reason for Optimality:** At every step, A* selects the node with the smallest $f(n)$ value (the sum of the actual path cost and the heuristic estimate). In this graph, the path $S \rightarrow A \rightarrow C \rightarrow G$ reaches the goal with the lowest total cost of **6**, making it the optimal solution.

Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.

Variables: WA, NT, SA, Q, NSW, V, T | Domain (each variable): {Red, Green, Blue}
Constraint: adjacent (connected) regions must not have the same colour. T has no shared border with any other region.



The variables are the states and territories of Australia.

$\{WA, NT, SA, Q, NSW, V, T\}$

Where:

- WA = Western Australia
- NT = Northern Territory
- SA = South Australia
- Q = Queensland
- NSW = New South Wales
- V = Victoria
- T = Tasmania

Domains

Each variable can take one of three colours.

$\{Red, Green, Blue\}$

Variable	Domain
WA	{Red, Green, Blue}
NT	{Red, Green, Blue}
SA	{Red, Green, Blue}
Q	{Red, Green, Blue}
NSW	{Red, Green, Blue}
V	{Red, Green, Blue}
T	{Red, Green, Blue}

Constraints

Adjacent states cannot have the same colour.

The constraints are:

- $WA \neq NT$
- $WA \neq SA$
- $NT \neq SA$
- $NT \neq Q$
- $SA \neq Q$
- $SA \neq NSW$

- $SA \neq V$
- $Q \neq NSW$
- $NSW \neq V$

Tasmania (T) has **no neighbouring state**, so it can be assigned any colour.

2. Backtracking Search (Manual Dry Run)

Colour Order Used

Red → Green → Blue

Variable Order

WA → NT → SA → Q → NSW → V → T

Step 1

Assign

WA=Red WA = Red WA=Red

Constraint Check

No previous assignment.

Valid ✓

Current Assignment

Variable Colour

WA Red

Step 2

Assign

NT=Red NT = Red NT=Red

Constraint Check

WA = Red

NT is adjacent to WA.

Red = Red ✗

Constraint violated.

Reject.

Try next colour.

Assign

NT=GreenNT = GreenNT=Green

Check

WA(Red) $\neq$ NT(Green)

Valid ✓

Current Assignment

Variable Colour

WA Red

NT Green

Step 3

Assign

SA=RedSA = RedSA=Red

Check constraints

SA adjacent to WA

Red = Red ✗

Constraint violated and Reject.

Try

SA=Green SA = Green SA=Green

Check

NT = Green

Green = Green ✗

Reject.

Try

SA=Blue SA = Blue SA=Blue

Check

- WA(Red) ≠ Blue ✓
- NT(Green) ≠ Blue ✓

Valid ✓

Current Assignment

Variable Colour

WA	Red
NT	Green
SA	Blue

Step 4

Assign

Q=Red Q = Red Q=Red

Check

Adjacent to NT

Green ≠ Red ✓

Adjacent to SA

Blue $\neq$ Red ✓

Valid ✓

Current Assignment

Variable Colour

WA	Red
NT	Green
SA	Blue
Q	Red

Step 5

Assign

NSW=RedNSW = RedNSW=Red

Constraint Check

Adjacent to Q

Red = Red ✗

Constraint violated.

Reject.

Try

NSW=GreenNSW = GreenNSW=Green

Check

Q(Red) $\neq$ Green ✓

SA(Blue) $\neq$ Green ✓

Valid ✓

Current Assignment

Variable Colour

WA	Red
NT	Green
SA	Blue
Q	Red
NSW	Green

Step 6

Assign

V=Red V = Red V=Red

Check

Adjacent to NSW

Green $\neq$ Red ✓

Adjacent to SA

Blue $\neq$ Red ✓

Valid ✓

Current Assignment

Variable Colour

WA	Red
NT	Green
SA	Blue
Q	Red
NSW	Green
V	Red

Step 7

Assign

T=RedT = RedT=Red

Tasmania has no neighbours.

No constraints.

Valid ✓

Final Assignment

Variable Colour

WA	Red
NT	Green
SA	Blue
Q	Red
NSW	Green
V	Red
T	Red

Constraint Checking Table

Step	Variable	Colour Tried	Constraint Check	Result
1	WA	Red	No previous constraint	✓
2	NT	Red	WA=Red	✗
	NT	Green	WA≠Green	✓
3	SA	Red	WA=Red	✗
	SA	Green	NT=Green	✗
	SA	Blue	Different from WA & NT	✓
4	Q	Red	Different from NT & SA	✓
5	NSW	Red	Q=Red	✗
	NSW	Green	Different from Q & SA	✓
6	V	Red	Different from NSW & SA	✓
7	T	Red	No neighbours	✓

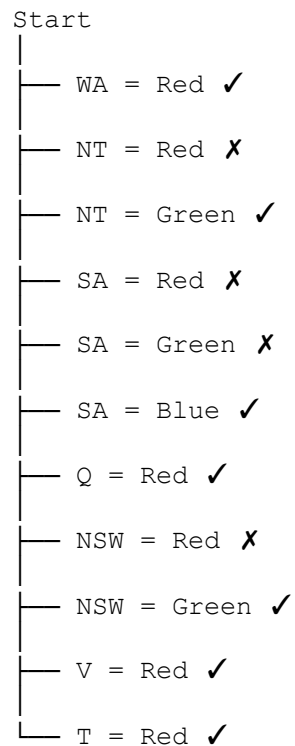
Backtracking Steps

The algorithm encountered conflicts while assigning colours and tried alternative colours before proceeding:

1. **NT = Red** → Conflict with **WA = Red** → Backtracked and assigned **NT = Green**.
2. **SA = Red** → Conflict with **WA = Red** → Rejected.
3. **SA = Green** → Conflict with **NT = Green** → Rejected.
4. **SA = Blue** → Satisfied all constraints.
5. **NSW = Red** → Conflict with **Q = Red** → Backtracked and assigned **NSW = Green**.

No further backtracking was required after these corrections.

Search Tree (Backtracking)



Final Solution

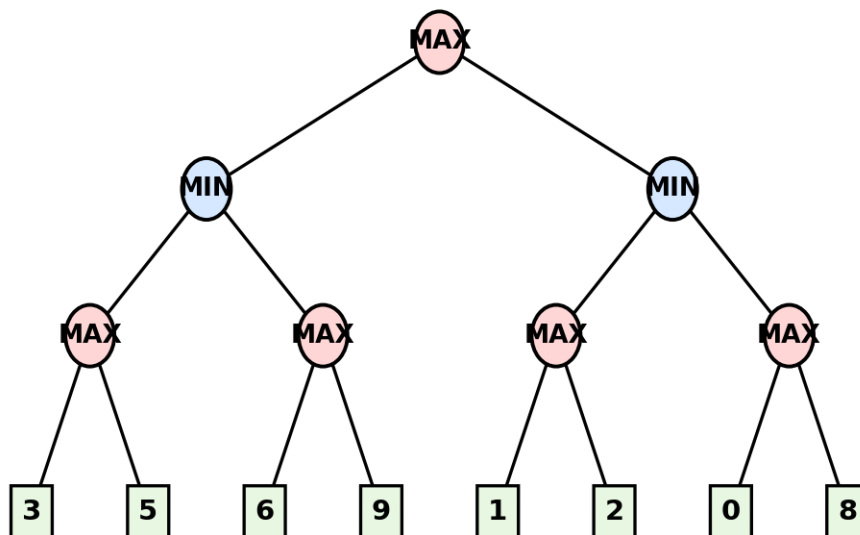
Colour Assignment:

- **WA = Red**
- **NT = Green**
- **SA = Blue**
- **Q = Red**

- NSW = **Green**
- V = **Red**
- T = **Red**

Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Step 1: Terminal (Leaf) Utility Values

The utility values at the leaf nodes are:

Left Subtree	Right Subtree
3	1
5	2
6	0
9	8

Step 2: Evaluate the MAX Nodes (Bottom-Up)

Since these are **MAX nodes**, each one selects the **maximum** value among its children.

MAX Node A

Children:

- 3
- 5

$$\max_{f_0}(3,5)=5 \quad \max(3,5)=5 \quad \max(3,5)=5$$

Backed-up value:

$$5 \boxed{5} 5$$

MAX Node B

Children:

- 6
- 9

$$\text{Max } (6,9)=9$$

$$\max(6,9)=9$$

$$\max(6,9)=9$$

Backed-up value:

$$9 \boxed{9} 9$$

MAX Node C

Children:

- 1
- 2

$$\max_{f_0}(1,2)=2 \quad \max(1,2)=2 \quad \max(1,2)=2$$

Backed-up value:

$$2 \boxed{2} 2$$

MAX Node D

Children:

- 0
- 8

$$\max_{f_0}(0,8)=8 \quad \max(0,8)=8 \quad \max(0,8)=8$$

Backed-up value:

$$8 \boxed{8}$$

Step 3: Evaluate the MIN Nodes

Each **MIN node** selects the **smallest** value among its MAX children.

Left MIN Node

Children:

- MAX A = 5
- MAX B = 9

$$\min_{f_0}(5,9)=5 \quad \min(5,9)=5 \quad \min(5,9)=5$$

Backed-up value:

$$5 \boxed{5}$$

Right MIN Node

Children:

- MAX C = 2
- MAX D = 8

$$\min_{f_0}(2,8)=2 \quad \min(2,8)=2 \quad \min(2,8)=2$$

Backed-up value:

$$2 \boxed{2}$$

Step 4: Evaluate the Root (MAX)

Root receives

- Left MIN = 5
- Right MIN = 2

Since root is MAX,

$\max(5,2)=5$

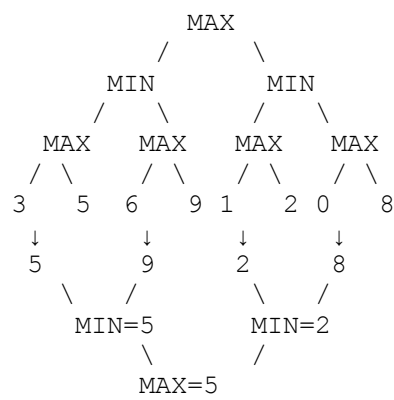
Therefore,

$\text{Root Value}=5$

Manual Calculation Table

Node	Children	Operation	Computation	Backed-up Value
MAX A	3, 5	MAX	$\max(3,5)$	5
MAX B	6, 9	MAX	$\max(6,9)$	9
MAX C	1, 2	MAX	$\max(1,2)$	2
MAX D	0, 8	MAX	$\max(0,8)$	8
MIN Left	5, 9	MIN	$\min(5,9)$	5
MIN Right	2, 8	MIN	$\min(2,8)$	2
Root MAX	5, 2	MAX	$\max(5,2)$	5

Tree Evaluation (Bottom-Up)



Order of Evaluation

The Minimax algorithm evaluates the tree from the bottom (leaf nodes) to the top (root):

1. Evaluate $\text{MAX}(3,5) = 5$
2. Evaluate $\text{MAX}(6,9) = 9$
3. Evaluate $\text{MAX}(1,2) = 2$
4. Evaluate $\text{MAX}(0,8) = 8$
5. Evaluate $\text{MIN}(5,9) = 5$
6. Evaluate $\text{MIN}(2,8) = 2$
7. Evaluate $\text{MAX}(5,2) = 5$

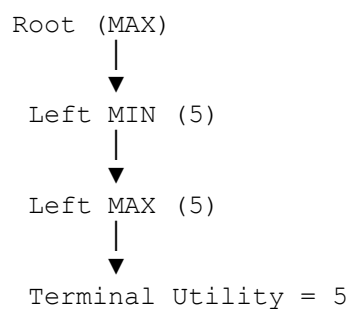
Best Move for MAX

At the root node, MAX compares:

- Left branch = 5
- Right branch = 2

Since $5 > 2$, MAX selects the **left branch**.

Final Game Path

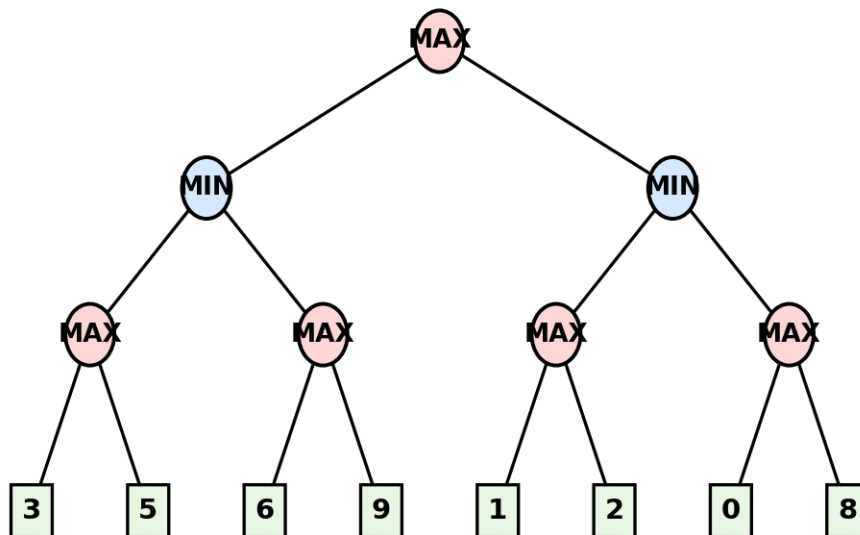


Final Answer

- **Leaf Utility Values:** 3, 5, 6, 9, 1, 2, 0, 8
- **Backed-up Values at MAX Nodes:** 5, 9, 2, 8
- **Backed-up Values at MIN Nodes:** 5, 2
- **Root (MAX) Value:** 5
- **Best Move for MAX:** Choose the left subtree

Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Initial Values

At the root node,

$$\alpha = -\infty, \beta = +\infty$$

Traversal is performed from **left to right**.

Step 1: Visit Left MIN Node

Current values

$$\alpha = -\infty, \beta = +\infty \quad \alpha = -\infty, \beta = +\infty$$

Evaluate Left MAX Node

Leaf nodes:

3 5

Current values

$$\alpha = -\infty, \beta = +\infty$$

Visit Leaf = 3

$$\alpha = \max_{f_0}(-\infty, 3) = 3$$

Visit Leaf = 5

$$\alpha = \max_{f_0}(3, 5) = 5$$

MAX returns

5

Update Left MIN Node

Current

$$\beta = \min_{f_0}(+\infty, 5) = 5$$

Now

$$\alpha = -\infty, \beta = 5$$

Evaluate Right MAX Node

Leaves

6 9

Current

$$\alpha = -\infty, \beta = 5$$

Visit Leaf = 6

$$\alpha = \max_{f_0}(-\infty, 6) = 6$$

Now

$$\alpha = 6, \beta = 5$$

Since

$$\alpha \geq \beta$$

Alpha-Beta condition is satisfied.

Therefore,

Leaf node 9 is pruned.

✗ Branch containing 9 is NOT evaluated.

MAX returns

6

Left MIN Node

Compare

$$\min(5, 6) = 5$$

Left MIN returns

5

Step 2: Update Root (MAX)

Root receives

555

Update alpha

$$\alpha = \max_{f_0}(-\infty, 5) = 5$$

Current values

$$\alpha=5, \beta=+\infty$$

Step 3: Visit Right MIN Node

Current values

$$\alpha=5, \beta=+\infty$$

Evaluate Left MAX Node

Leaves

1 2

Current

$$\alpha=5, \beta=+\infty$$

Visit Leaf 1

Maximum =1

Visit Leaf 2

Maximum =2

Return

$$2 \boxed{2}$$

Update Right MIN Node

$$\beta = \min\{\overset{f_0}{\dots}\}(+\infty, 2) = 2$$

Now

$$\alpha=5, \beta=2$$

Since

$$\alpha \geq \beta$$

Pruning condition is satisfied.

Therefore,

the **entire right MAX subtree (0,8)** is pruned.

✗ Leaf 0 not evaluated.

✗ Leaf 8 not evaluated.

MIN returns

$\boxed{2}$

Step 4: Root MAX

Root compares

Left =5

Right =2

$\max_{f_0}(5,2)=5$

Root returns

$\boxed{5}$

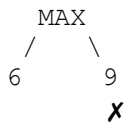
Alpha-Beta Dry Run Table

Step	Node	α	β	Action
1	Root MAX	$-\infty$	$+\infty$	Start
2	Left MIN	$-\infty$	$+\infty$	Visit
3	Left MAX	$-\infty$	$+\infty$	Evaluate 3,5 $\rightarrow$ Return 5
4	Left MIN	$-\infty$	5	β updated to 5
5	Right MAX	$-\infty$	5	First child =6
6	Right MAX	6	5	$\alpha \geq \beta \rightarrow$ Prune 9
7	Root MAX	5	$+\infty$	α updated to 5
8	Right MIN	5	$+\infty$	Visit
9	Left MAX	5	$+\infty$	Evaluate 1,2 $\rightarrow$ Return 2
10	Right MIN	5	2	$\alpha \geq \beta$
11	Right MAX	5	2	Prune 0 and 8
12	Root MAX	5	$+\infty$	Final value =5

Pruned Branches

First Pruning

At the Left MIN subtree



Reason:

After evaluating **6**,

$$\alpha=6, \beta=5 \quad \alpha=6, \beta=5$$

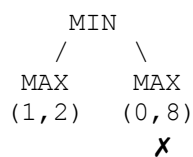
Since

$$6 \geq 5 \quad 6 \geq 5$$

the remaining child (**9**) cannot influence the MIN node's decision, so it is pruned.

Second Pruning

At the Right MIN subtree



After obtaining

$$\beta=2$$

Root already has

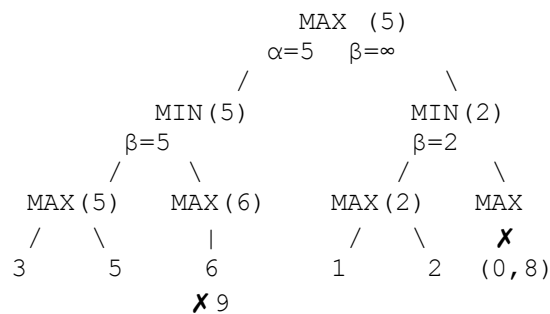
$$\alpha=5$$

Since

$$5 \geq 5$$

the second MAX subtree (**0,8**) cannot improve the result and is completely pruned.

Final Evaluated Tree



Final Best Move

The root (MAX) compares:

- Left subtree = 5
- Right subtree = 2

Since

$5 > 2$

the **best move** for MAX is to choose the **left subtree**.

Final Answer

- **Algorithm:** Minimax with Alpha-Beta Pruning
- **Initial Values:** $\alpha = -\infty$, $\beta = +\infty$
- **Pruned Branches:**
 - Leaf node **9** (because $\alpha = 6 \geq \beta = 5$)
 - Entire subtree **{0, 8}** (because $\alpha = 5 \geq \beta = 2$)
- **Final Backed-up Value at Root:** 5
- **Best Move for MAX:** Choose the left subtree

Code of Conduct:

I DHANUSH S (192524317) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S DHANUSH (192524317)

Dry Run Evaluation**RUBRICS FOR CALCULATION**

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	10
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	10
Accuracy of Computations	All computed values are	Minor calculation	Multiple calculation	Calculations are mostly	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
(g(n), h(n), f(n) values, minimax backed-up values, or α - β updates)	accurate and consistent throughout the dry run.	errors are present but do not significantly affect the final outcome.	errors are present that affect the correctness of intermediate steps.	incorrect or missing.	
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	10
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	10
				Total	50